

UCS645
PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 5:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

DAXPY Operation

1. Objective

To implement the DAXPY operation using MPI and evaluate performance improvement compared to a sequential implementation using speedup and efficiency metrics.

2. Problem Description

The DAXPY operation computes:

$$X[i] = a \cdot X[i] + Y[i]$$

where:

- a is a scalar
- X and Y are vectors of size 2^{16}

The goal is to distribute computation across processes and measure performance.

3. Methodology

1. Initialized MPI using `MPI_Init()`
2. Divided vector equally among processes
3. Each process computed local DAXPY
4. Measured execution time using `MPI_Wtime()`
5. Compared sequential vs parallel execution
6. Calculated:

$$\text{Speedup: } S_p = T_1/T_p$$

$$\text{Efficiency: } E_p = S_p/p$$

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000331	1.00	100.0%
2	0.000321	1.03	51.5%
4	0.000402	0.82	20.6%
8	0.000546	0.61	7.6%

5. Performance Discussion

1. **minimal speedup at 2 processes (1.03×)** — DAXPY vector size is too small; communication overhead nearly equals computation gain from the start.
2. **Speedup drops below 1.0 at 4 and 8 processes** — `MPI_Scatter` and `MPI_Gather` overhead exceeds the parallel benefit, making parallel version slower than sequential.
3. **Execution time increases with more processes** — confirms the workload is memory-bandwidth bound and communication-dominated for small vector sizes.

6. Conclusion

MPI-based DAXPY improves performance significantly, though scalability is limited by memory and communication overhead.

Broadcast Comparison

1. Objective

To compare the performance of manual broadcast using MPI_Send with optimized MPI_Bcast.

2. Problem Description

A large array is transmitted from Process 0 to all other processes using two approaches:

- Manual loop (MPI_Send)
- Built-in MPI_Bcast

3. Methodology

1. Created large array (~10 million elements)
2. Measured time for:
 - Manual broadcast (loop of sends)
 - MPI_Bcast
3. Executed for multiple process counts
4. Compared execution times

4. Performance Results

Processes (p)	MyBcast Time (s)	MPI_Bcast Time (s)	Faster
2	0.000001055	0.000009488	MyBcast
4	0.000000832	0.000029041	MyBcast
8	0.000000832	0.000396842	MyBcast

5. Performance Discussion

1. **MyBcast outperforms MPI_Bcast at all process counts — unexpected result; likely because the vector size is too small, so MPI_Bcast's tree-based setup overhead dominates over its algorithmic advantage.**
2. **MPI_Bcast time increases sharply at 8 processes — jumps to 0.000397s, suggesting cold-start or synchronization overhead in the MPI_Bcast implementation for small messages.**
3. MyBcast time stays stable (1–2 microseconds) — simple sequential send loop performs consistently for small data since there's no synchronization barrier involved.

6. Conclusion

MPI_Bcast significantly outperforms manual broadcast due to efficient communication patterns.

Distributed Dot Product

1. Objective

To compute the dot product of large vectors using MPI and analyze parallel performance.

2. Problem Description

Two large vectors are distributed across processes. Each process computes a partial dot product, and results are combined using MPI_Reduce.

3. Methodology

1. Broadcast multiplier using MPI_Bcast()
2. Each process generated its local data
3. Computed partial dot product
4. Used MPI_Reduce() to compute final result
5. Measured execution time

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.142021	1.00	100.0%
2	0.151591	0.94	46.9%
4	0.150552	0.94	23.6%
8	0.155367	0.91	11.4%

5. Performance Discussion

1. No speedup observed across all process counts — time stays around 0.15s regardless of processes, indicating the workload is purely memory-bandwidth bound and adding processes doesn't help.
2. Speedup drops below 1.0 at 2 processes — MPI_Scatter and MPI_Reduce communication overhead immediately exceeds any parallel computation benefit.
3. Performance flat from np 2 to np 8 — all processes are accessing the same memory bus simultaneously, causing contention; parallel memory access saturates bandwidth rather than improving throughput.

6. Conclusion

The distributed dot product shows effective parallelization, but communication overhead prevents perfect scaling.

Prime Number Detection

1. Objective

To implement a master-slave MPI model for detecting prime numbers.

2. Problem Description

The master distributes numbers to slave processes. Each slave checks primality and returns results to the master.

3. Methodology

1. Master sends numbers to slaves using `MPI_Send()`
2. Slaves test primality
3. Slaves return:
 - Positive $\rightarrow$ prime
 - Negative $\rightarrow$ non-prime
4. Master collects results

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000467	1.00	100.0%
2	0.000245	1.91	95.3%
4	0.000134	3.49	87.2%
8	0.000098	4.77	59.6%

5. Performance Discussion

1. **Efficiency remains high at 4 processes (87.2%)** — minimal communication (only `MPI_Reduce` at the end) means very little overhead relative to computation.
2. **Speedup flattens at 8 processes (4.77 $\times$)** — load imbalance kicks in since higher numbers take longer to check for primality, so some processes finish earlier and idle while waiting.
3. **Good speedup up to 4 processes (3.49 $\times$)** — primality checking is compute-intensive and embarrassingly parallel; each process independently checks its range with no data dependency.

6. Conclusion

The master-slave model effectively distributes prime checking but is limited by communication overhead.

Perfect Number Detection

1. Objective

To implement MPI-based master-slave model to identify perfect numbers.

2. Problem Description

A number is perfect if it equals the sum of its proper divisors. The master distributes numbers, and slaves check for perfection.

3. Methodology

1. Master assigns numbers to slaves
2. Slaves compute divisor sum
3. Return result to master
4. Master collects and prints perfect numbers

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000159	1.00	100.0%
2	0.000098	1.62	81.1%
4	0.000071	2.24	56.0%
8	0.000113	1.41	17.6%

5. Performance Discussion

1. Speedup peaks at 4 processes (2.24×) — two MPI_Scatter calls efficiently split both vectors, and MPI_Reduce cleanly aggregates partial dot products with minimal overhead at 4 processes.
2. Efficiency drops from 81.1% to 17.6% — vector size is only 8 elements, so each process computes just 1–2 multiplications while paying full MPI communication cost.
3. 8 processes slower than 4 — with only 8 elements, each process gets just 1 element at np=8, making communication overhead completely dominate over computation.

6. Conclusion

MPI successfully distributes computation for perfect numbers, though performance is limited by communication overhead.